NAME: ABDULLAH **ROLL NO: 044**

CLASS: BSAI **SECTION: 3A**

LAB TASK: 2

TASK:1

PEAS:

1 Delivery Robot

Performance:

Deliver the package to the right address in a timely manner.

No accidents involving people, animals or vehicles.

Keep the package safe along the route.

Have enough battery to complete the journey.

Follow the rules for foot paths, roads etc.

Environment:

Roads, Foot paths, Gates, Building Entrances.

People, Children, Bikes, Rickshaws, Cars.

Actuators:

Motors & Steering.

Braking.

Package box with Electric Lock.

Horn, Headlights, Indicators.

Tiny screen & Speaker for communicating with recipient.

Sensors:

Cameras (Front & Back).

Ultrasonic & Lidar sensor for distance.

GPS.

Wheel Speed Sensor & Gyroscope.

Battery Meter.

Weight Sensor (Inside Package Box).

Trade-off:

The customer wants the package delivered quickly. Therefore the robot will need to move fast. The problem with speed is it will take longer to stop and could cause injury if a child suddenly appears in front of it. The battery will also run out quicker. Safety should be the priority so the robot will move slower in busy areas. This will cause the delivery to be a little late.

Assumptions:

Map of surroundings is already stored in the robot.

Mostly used during the day on foot paths. Not during heavy rain.

There is internet so if the robot gets stuck an operator can assist from afar.

Only one package is delivered each time. The recipient opens using an OTP sent to their phone.

2 Student Support Agent (LLM):

Performance:

Give correct answers based on university rules.

Quick & polite responses.

Solve the issue without the student having to ask multiple times.

Pass difficult cases to real staff.

No private information shown of students.

Environment:

Student asking questions via the university app/website chat.

University Handbook, Fee Structure, Date Sheet & Notices.

Student Portal, LMS, Timetable Data.

Teachers, Admin Office & Helpdesk staff.

Actuators:

Type response into the chat.

Create Support Ticket.

Send email/notification to relevant department.

Pass chat to human staff.

Sensors:

Student message & previous chat.

Login information (Student ID, Programme, Semester).

Search results from document.

Information from tools (Timetable, Fee Status etc.).

Trade-off:

If the agent answers all questions it will provide a quick response. However there is a risk of providing incorrect answers. Example: Incorrect deadline for fees. If all small questions are passed to staff then the answers will be safer but staff will be overwhelmed. The agent should answer simple repeating questions like timetable and deadlines but pass serious issues like marks, fee extension & complaints.

Assumptions:

Student must log in so the agent will know who they are chatting to.

The agent can only view information. It cannot change marks, registration or fees.

It can only view the logged in students information.

Documents searched are updated by the university occasionally.

Task 2:

Classifying Environments

System

Observable

Deterministic/Stochastic

Reason

Delivery robot

Partially Observable

Stochastic

Robot can only view local area due to sensors. Road, Wheels & People can be unpredictable.

Student Support Agent

Partially Observable

Stochastic

Cannot view what the student really means. Responses and search results can vary.

1 Delivery Robot

Partially Observable: The robot knows certain things for example the battery %, how fast it is moving & how far away the object in front is. However it does not know everything. It cannot see children running behind parked cars, doesn't know if gates are closed until it reaches them & cannot tell if the road is slippery due to rain. It must work with incomplete information.

Stochastic: Some actions will produce the same outcome. For example if the recipient enters the correct OTP the box lock will open every time. However many actions will not.

2 Student Support Agent

Partially Observable: The agent can view the students message, previous chat and student ID from login. It cannot see what the student really wants. If a student says "my result is wrong" the agent will not know if they mean their marks, GPA or there is an error in the portal. It cannot be completely sure the handbook it is searching is the most recent version.

Stochastic: When asking about fee status using the fee tool it will produce the same answer as it is pulling from the database. However the response can be unpredictable. The language model will change the wording each time, the search results will pull up a different page and the student may be happy with the answer or ask again. The agent will not know this beforehand.

3 Real World vs Simulated Environment

Both systems will be partially observable & stochastic in the real world. When using a simulator we can simplify the environment. With the delivery robot a simulator can be a small game level with a straight corridor and 3 set obstacles. Each movement will make the robot move one step. This makes it fully observable & deterministic. With the support agent a simulator can be a testing program with 15 set questions and answers. This makes it almost deterministic and almost fully observable.

Depending on your assumptions the environment can be different. My answers in the table are based on the real world. Using real people, sensors & the university system. However if you remove the noise, hidden information & random users in the simulator the answer will differ. An agent that works well in a simple simulator can fail in the real world as there will be surprises.

Task 3:

Mode Memory Controller

Rules Used:

No one in room: Mode OFF regardless of temperature.

Someone in room & temp > 27: Mode COOL.

Someone in room & temp < 21: Mode HEAT.

Else: Mode OFF

The temperature thresholds were decided by me. The last_mode starts as None so the first command will always be sent.

```
# Mode memory controller for room (AC/heater)
last_mode = None                # Stores last mode sent, initially nothing is sent.
def decide_mode (temp, people_inside):
global last_mode
# If no one in room everything is off
if people_inside == False:
new_mode = "OFF"
elif temp > 27:
new_mode = "COOL"
elif temp < 21:
new_mode = "HEAT"
else:
new_mode = "OFF"
# Only send command if new mode is different to last mode
if new_mode != last_mode:
send_command = True
else:
send_command = False
last_mode = new_mode
return new_mode, send_command
# Test cases (Temp, People Inside)
test_list = [
(25, True),
```

```
(25, True),
(29, True),
(30, True),
(27, True),
(21, True),
(19, True),
(19, True),
(19, False),
(32, False),
(31, True)
]
print("No | Last mode | Temp | People | New Mode | Command")
count = 1
for temp, people in test_list:
old = last_mode
mode, send = decide_mode(temp, people)
if send == True:
result = "SEND"
else:
result = "NO COMMAND"
print(count,"|", old,"|", temp,"|", people,"|", mode,"|", result)
count += 1
```

Output Table:

Each row represents one input.

No.	Last mode	Temp	People	New Mode	Command	Case Tested
1	None	25	Yes	OFF	SEND	First input, no info stored.
2	OFF	25	Yes	OFF	NO COMMAND	Same input.
3	OFF	29	Yes	COOL	SEND	Changed from OFF to COOL.
4	COOL	30	Yes	COOL	NO COMMAND	Same input.

5 COOL 27 Yes OFF SEND Exactly 27 so mode OFF. Still sent as changed from COOL.

6 OFF 21 Yes OFF NO COMMAND Exactly 21 so mode OFF. No command.

7 OFF 19 Yes HEAT SEND Changed from OFF to HEAT.

8 HEAT 19 Yes HEAT NO COMMAND Same input.

9 HEAT 19 No OFF SEND No one in room.

10 OFF 32 No OFF NO COMMAND No one in room & very hot.

11 OFF 31 Yes COOL SEND Person came back into hot room.

GITHUB LINK:

https://github.com/abdullahrehman957-lgtm/AI-LAB-TASK
